

# COMPSYS302: Python

## 7. Exception

**Ho Seok AHN**

hs.ahn@auckland.ac.nz



**THE UNIVERSITY  
OF AUCKLAND**

**FACULTY OF ENGINEERING**

# 7.1 Exceptions

## □ Exceptions

- Python uses **exceptions** to communicate errors and anomalies.
- An exception is an object that indicates an error or anomalous condition.
- When Python detects an error, it raises an exception; that is, it signals the occurrence of an anomalous condition by passing an exception object to the exception-propagation mechanism.
- Handling an exception means receiving the exception object from the propagation mechanism and performing whatever actions are needed to deal with the anomalous situation.

## 7.2 The try Statement

### □ The try Statement

- The **try** statement provides Python's exception-handling mechanism.
- It is a compound statement that can take one of two different forms.
- The first form is:

```
try:  
    statement(s)  
except [expression [, target]]:  
    statement(s)
```

- Ex) 7\_01\_TryExcept.py

```
c = 5 / 0
```

(result)

Traceback (most recent call last):

File "7\_01\_TryExcept.py", line 1, in <module>

```
c = 5 / 0
```

ZeroDivisionError: integer division or modulo by zero

```
try:
```

```
c = 5 / 0
```

```
except:
```

```
    print ("caught an exception")
```

(result)

caught an exception

## 7.2 The try Statement

### □ The try Statement

- The second form is:

```
try:  
    statement(s)  
finally:  
    statement(s)
```

- This form has exactly one finally clause.

- Ex) 7\_02\_TryFinally.py

```
try:  
    c = 5 / 0  
finally:  
    print ("anyway execute")
```

(result)

anyway execute

Traceback (most recent call last):

File "7\_02\_TryFinally.py", line 2, in <module>

c = 5 / 0

ZeroDivisionError: integer division or modulo by zero

## 7.2 The try Statement

### □ The try Statement

- You can use both form as follows:

```
try:  
    statement(s)  
except [expression [, target]]:  
    statement(s)  
finally:  
    statement(s)
```

- Ex) 7\_03\_TryExceptFinally.py

```
try:  
    c = 5 / 0  
except:  
    print ("caught an exception")  
finally:  
    print ("anyway execute")
```

(result)  
caught an exception  
anyway execute

## 7.3 The raise Statement

### □ The raise Statement

- You can use the **raise** statement to raise an exception explicitly.
- **raise** is a simple statement with the following syntax:

`raise [expression1 [, expression2]]`

- Ex) 7\_04\_Raise.py

```
def RaiseExample(a):  
    if int(a) < 0:  
        raise Exception ("the number is negative")  
    else:  
        print ("your age is", a)  
  
def RaiseExample2(a):  
    if int(a) < 5:  
        raise Exception ("you're too young for schooling")  
    else:  
        print ("Enjoy your schooling")
```

```
try:  
    a = input("enter your age: ")  
    RaiseExample(a)  
    RaiseExample2(a)  
except Exception as e:  
    print ("exception: %s" %e)  
  
(result)  
enter your age: 3  
your age is 3  
exception: you're too young for schooling
```